Computational Physics Ex_4

Student ID: 2024141230044

Name: Fanyi

Problem 4

For the quadratic equation

$$1.22x^2 + 3.34x + 2.28 = 0$$

use the standard quadratic formula to compute its roots and examine the cancellation error.

In addition, compute

$$(x-1)^7$$

and its expanded form

$$x^7 - 7x^6 + 21x^5 - 35x^4 + 35x^3 - 21x^2 + 7x - 1$$

in single, double, and quad precision, and compare these calculations by zooming in near $x \approx 1$.

Contents

1 Algorithm Principle 3

2 Program Code 4

3 Execution Results 7

4 Result Analysis 9

1 Algorithm Principle

Quadratic Formula

For a general quadratic equation

$$ax^2 + bx + c = 0$$

the standard quadratic formula is

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

In this exercise,

$$a = 1.22, \qquad b = 3.34, \qquad c = 2.28$$

so the discriminant is

$$\Delta = b^2 - 4ac = 3.34^2 - 4 \times 1.22 \times 2.28 = 0.0292$$

Because $b > 0$, the computation of

$$\frac{-b + \sqrt{\Delta}}{2a}$$

involves subtracting two nearby numbers. As a result, this root is more vulnerable to cancellation error.

In this report, the standard quadratic formula is used as required by the problem. The stable form is included only as a comparison, so that the cancellation error can be seen more clearly.

To reduce this problem, we can use the stable form

$$q = -\frac{1}{2}\left(b + \text{sign}(b)\sqrt{\Delta}\right)$$

and then compute

$$x_1 = \frac{q}{a}, \qquad x_2 = \frac{c}{q}$$

This avoids the harmful subtraction and improves numerical stability.

Factored Form and Expanded Form

The two expressions in the problem are mathematically identical:

$$(x-1)^7 = x^7 - 7x^6 + 21x^5 - 35x^4 + 35x^3 - 21x^2 + 7x - 1$$

However, their numerical behavior is not the same.

When $x \approx 1$, the factored form $(x-1)^7$ first computes the small quantity $(x-1)$ and then raises it to the seventh power, so the evaluation path is relatively direct. In contrast, in the expanded form each individual term is close to 1, while the final result is extremely close to 0. Therefore, many large positive and negative terms cancel each other, and a large number of significant digits can be lost.

This problem therefore uses the comparison between the *factored form* and the *expanded form* to demonstrate a typical floating-point cancellation error.

2 Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <float.h>
#include <quadmath.h>

static void print_q(const char *label, __float128 x) {
    char buf[128];
    quadmath_snprintf(buf, sizeof(buf), "%.36Qg", x);
    printf("%-24s = %s\n", label, buf);
}

static void sort2f(float *x, float *y) {
    if (*x > *y) {
        float t = *x; *x = *y; *y = t;
    }
}

static void sort2d(double *x, double *y) {
    if (*x > *y) {
```

```
        double t = *x; *x = *y; *y = t;
    }
}

static void sort2q(__float128 *x, __float128 *y) {
    if (*x > *y) {
        __float128 t = *x; *x = *y; *y = t;
    }
}

/* Round to a given number of decimal significant digits. */
static double round_sig(double x, int sig) {
    if (x == 0.0) {
        return 0.0;
    }

    double ax = fabs(x);
    double e = floor(log10(ax));
    double scale = pow(10.0, sig - 1 - e);
    return round(x * scale) / scale;
}

/* Simulate fl(x op y) in 3-significant-digit decimal arithmetic.
   */
static double fl_add(double x, double y) { return round_sig(x + y,
    3); }
static double fl_sub(double x, double y) { return round_sig(x - y,
    3); }
static double fl_mul(double x, double y) { return round_sig(x * y,
    3); }

static void discriminant_three_digit_demo(void) {
    double a = 1.22, b = 3.34, c = 2.28;
    double exact_disc = b * b - 4.0 * a * c;

    double b2_exact = b * b;
    double b2_fl = round_sig(b2_exact, 3);

    /* Compute 4ac with rounding after each multiplication. */
    double foura_fl = fl_mul(4.0, a);
```

```
    double fourac_fl = fl_mul(foura_fl, c);
    double fourac_exact = 4.0 * a * c;

    double disc_fl = fl_sub(b2_fl, fourac_fl);
    double abs_err = fabs(disc_fl - exact_disc);
    double rel_err = abs_err / fabs(exact_disc);

    printf("\n[3-significant-digit arithmetic demo for b^2 - 4ac]\
   n");
    printf("a = %.2f, b = %.2f, c = %.2f\n", a, b, c);
    printf("exact b^2                 = %.10g\n", b2_exact);
    printf("fl(b^2)                   = %.10g\n", b2_fl);
    printf("exact 4ac                 = %.10g\n", fourac_exact);
    printf("fl(4a)                    = %.10g\n", foura_fl);
    printf("fl(fl(4a)*c) = fl(4ac)    = %.10g\n", fourac_fl);
    printf("exact b^2 - 4ac           = %.10g\n", exact_disc);
    printf("fl(fl(b^2) - fl(4ac))     = %.10g\n", disc_fl);
    printf("absolute error            = %.10g\n", abs_err);
    printf("relative error            = %.10g  (%.2f%%)\n",
   rel_err, 100.0 * rel_err);

    printf("\nInterpretation:\n");
    printf("  b^2  = 11.1556  -> 11.2  (3 significant digits)\n");
    printf("  4ac  = 11.1264  -> 11.1  (3 significant digits)\n");
    printf("  11.2 - 11.1 = 0.1, while the exact value is 0.0292\n
   ");
    printf("  This is a classic cancellation error.\n");
}

static float direct7f(float x) {
    float t = x - 1.0f;
    float t2 = t * t;
    float t4 = t2 * t2;
    return t4 * t2 * t;
}

static double direct7d(double x) {
    double t = x - 1.0;
    double t2 = t * t;
    double t4 = t2 * t2;
```

```
    return t4 * t2 * t;
}

static __float128 direct7q(__float128 x) {
    __float128 t = x - 1.0Q;
    __float128 t2 = t * t;
    __float128 t4 = t2 * t2;
    return t4 * t2 * t;
}

/* Expanded polynomial written explicitly (not Horner),
 * so the cancellation is easier to observe near x = 1.
 */
static float expanded7f(float x) {
    float x2 = x * x;
    float x3 = x2 * x;
    float x4 = x3 * x;
    float x5 = x4 * x;
    float x6 = x5 * x;
    float x7 = x6 * x;
    return x7 - 7.0f * x6 + 21.0f * x5 - 35.0f * x4 +
           35.0f * x3 - 21.0f * x2 + 7.0f * x - 1.0f;
}

static double expanded7d(double x) {
    double x2 = x * x;
    double x3 = x2 * x;
    double x4 = x3 * x;
    double x5 = x4 * x;
    double x6 = x5 * x;
    double x7 = x6 * x;
    return x7 - 7.0 * x6 + 21.0 * x5 - 35.0 * x4 +
           35.0 * x3 - 21.0 * x2 + 7.0 * x - 1.0;
}

static __float128 expanded7q(__float128 x) {
    __float128 x2 = x * x;
    __float128 x3 = x2 * x;
    __float128 x4 = x3 * x;
    __float128 x5 = x4 * x;
```

```
    __float128 x6 = x5 * x;
    __float128 x7 = x6 * x;
    return x7 - 7.0Q * x6 + 21.0Q * x5 - 35.0Q * x4 +
           35.0Q * x3 - 21.0Q * x2 + 7.0Q * x - 1.0Q;
}

static void quadratic_float(void) {
    float a = 1.22f, b = 3.34f, c = 2.28f;
    float disc = b * b - 4.0f * a * c;
    float s = sqrtf(disc);

    float r1_std = (-b + s) / (2.0f * a);
    float r2_std = (-b - s) / (2.0f * a);

    float q = -0.5f * (b + (b >= 0.0f ? s : -s));
    float r1_stable = q / a;
    float r2_stable = c / q;

    sort2f(&r1_std, &r2_std);
    sort2f(&r1_stable, &r2_stable);

    printf("\n[float]\n");
    printf("discriminant              = %.9g\n", disc);
    printf("standard root 1           = %.9g\n", r1_std);
    printf("standard root 2           = %.9g\n", r2_std);
    printf("stable   root 1           = %.9g\n", r1_stable);
    printf("stable   root 2           = %.9g\n", r2_stable);
}

static void quadratic_double(void) {
    double a = 1.22, b = 3.34, c = 2.28;
    double disc = b * b - 4.0 * a * c;
    double s = sqrt(disc);

    double r1_std = (-b + s) / (2.0 * a);
    double r2_std = (-b - s) / (2.0 * a);

    double q = -0.5 * (b + (b >= 0.0 ? s : -s));
    double r1_stable = q / a;
    double r2_stable = c / q;
```

```
    sort2d(&r1_std, &r2_std);
    sort2d(&r1_stable, &r2_stable);

    printf("\n[double]\n");
    printf("discriminant              = %.17g\n", disc);
    printf("standard root 1           = %.17g\n", r1_std);
    printf("standard root 2           = %.17g\n", r2_std);
    printf("stable   root 1           = %.17g\n", r1_stable);
    printf("stable   root 2           = %.17g\n", r2_stable);
}

static void quadratic_quad(void) {
    __float128 a = 1.22Q, b = 3.34Q, c = 2.28Q;
    __float128 disc = b * b - 4.0Q * a * c;
    __float128 s = sqrtq(disc);

    __float128 r1_std = (-b + s) / (2.0Q * a);
    __float128 r2_std = (-b - s) / (2.0Q * a);

    __float128 q = -0.5Q * (b + (b >= 0.0Q ? s : -s));
    __float128 r1_stable = q / a;
    __float128 r2_stable = c / q;

    sort2q(&r1_std, &r2_std);
    sort2q(&r1_stable, &r2_stable);

    printf("\n[quad: __float128]\n");
    print_q("discriminant", disc);
    print_q("standard root 1", r1_std);
    print_q("standard root 2", r2_std);
    print_q("stable   root 1", r1_stable);
    print_q("stable   root 2", r2_stable);
}

static void sample_point_report(void) {
    float xf = 1.0f + 1.0e-3f;
    double xd = 1.0 + 1.0e-9;
    __float128 xq = 1.0Q + 1.0e-12Q;
```

```
    printf("\nRepresentative sample points near x = 1:\n");
    printf("float  : x = %.9g, direct = %.9g, expanded = %.9g\n",
           xf, direct7f(xf), expanded7f(xf));
    printf("double : x = %.17g, direct = %.17g, expanded = %.17g\n
   ",
           xd, direct7d(xd), expanded7d(xd));

    char xb[128], d1[128], d2[128];
    quadmath_snprintf(xb, sizeof(xb), "%.36Qg", xq);
    quadmath_snprintf(d1, sizeof(d1), "%.36Qg", direct7q(xq));
    quadmath_snprintf(d2, sizeof(d2), "%.36Qg", expanded7q(xq));
    printf("quad   : x = %s, direct = %s, expanded = %s\n", xb, d1
   , d2);
}

static void write_csv(const char *filename, long n, long double
   left, long double right) {
    FILE *fp = fopen(filename, "w");
    if (!fp) {
        perror(filename);
        exit(EXIT_FAILURE);
    }

    fprintf(fp, "x,float_direct,float_expanded,double_direct,
   double_expanded,quad_direct,quad_expanded\n");

    for (long i = 0; i < n; ++i) {
        long double xl = left + (right - left) * (long double)i /
   (long double)(n - 1);

        float xf = (float)xl;
        double xd = (double)xl;
        __float128 xq = (__float128)xl;

        char qd[128], qe[128];
        quadmath_snprintf(qd, sizeof(qd), "%.36Qg", direct7q(xq));
        quadmath_snprintf(qe, sizeof(qe), "%.36Qg", expanded7q(xq)
   );

        fprintf(fp,
```

```
                "%.21Lg,%.9g,%.9g,%.17g,%.17g,%s,%s\n",
                xl,
                direct7f(xf), expanded7f(xf),
                direct7d(xd), expanded7d(xd),
                qd, qe);
    }

    fclose(fp);
}

static void write_gnuplot_script(void) {
    FILE *gp = fopen("plot.gp", "w");
    if (!gp) {
        perror("plot.gp");
        exit(EXIT_FAILURE);
    }

    fprintf(gp,
        "set datafile separator ','\n"
        "set term pngcairo size 1400,900\n"
        "set grid\n"
        "set key outside\n"
        "\n"
        "set output 'zoom1_float.png'\n"
        "set title 'Zoom 1: x in [0.99, 1.01] (float)'\n"
        "plot 'poly_zoom1.csv' using 1:2 with lines title '(x-1)
   ^7', \\\n"
        "     'poly_zoom1.csv' using 1:3 with lines title '
   expanded'\n"
        "\n"
        "set output 'zoom1_double.png'\n"
        "set title 'Zoom 1: x in [0.99, 1.01] (double)'\n"
        "plot 'poly_zoom1.csv' using 1:4 with lines title '(x-1)
   ^7', \\\n"
        "     'poly_zoom1.csv' using 1:5 with lines title '
   expanded'\n"
        "\n"
        "set output 'zoom1_quad.png'\n"
        "set title 'Zoom 1: x in [0.99, 1.01] (quad)'\n"
        "plot 'poly_zoom1.csv' using 1:6 with lines title '(x-1)
```

```
   ^7', \\\n"
        "     'poly_zoom1.csv' using 1:7 with lines title '
   expanded'\n"
        "\n"
        "set output 'zoom2_float.png'\n"
        "set title 'Zoom 2: x in [0.9999, 1.0001] (float)'\n"
        "plot 'poly_zoom2.csv' using 1:2 with lines title '(x-1)
   ^7', \\\n"
        "     'poly_zoom2.csv' using 1:3 with lines title '
   expanded'\n"
        "\n"
        "set output 'zoom2_double.png'\n"
        "set title 'Zoom 2: x in [0.9999, 1.0001] (double)'\n"
        "plot 'poly_zoom2.csv' using 1:4 with lines title '(x-1)
   ^7', \\\n"
        "     'poly_zoom2.csv' using 1:5 with lines title '
   expanded'\n"
        "\n"
        "set output 'zoom2_quad.png'\n"
        "set title 'Zoom 2: x in [0.9999, 1.0001] (quad)'\n"
        "plot 'poly_zoom2.csv' using 1:6 with lines title '(x-1)
   ^7', \\\n"
        "     'poly_zoom2.csv' using 1:7 with lines title '
   expanded'\n");

    fclose(gp);
}

int main(void) {
    printf("Quadratic equation: 1.22 x^2 + 3.34 x + 2.28 = 0\n");

    discriminant_three_digit_demo();

    quadratic_float();
    quadratic_double();
    quadratic_quad();

    sample_point_report();

    write_csv("poly_zoom1.csv", 4001, 0.99L, 1.01L);
```

```
    write_csv("poly_zoom2.csv", 4001, 0.9999L, 1.0001L);
    write_gnuplot_script();

    printf("\nGenerated files:\n");
    printf("  poly_zoom1.csv   (x in [0.99,   1.01])\n");
    printf("  poly_zoom2.csv   (x in [0.9999, 1.0001])\n");
    printf("  plot.gp          (gnuplot script)\n");
    printf("\nRun: gnuplot plot.gp\n");

    return 0;
}
```

3 Execution Results

According to the actual output of the final submitted program `ex4.c`, the results are as follows:

```
Quadratic equation: 1.22 x^2 + 3.34 x + 2.28 = 0

[3-significant-digit arithmetic demo for b^2 - 4ac]
a = 1.22, b = 3.34, c = 2.28
exact b^2                 = 11.1556
fl(b^2)                   = 11.2
exact 4ac                 = 11.1264
fl(4a)                    = 4.88
fl(fl(4a)*c) = fl(4ac)    = 11.1
exact b^2 - 4ac           = 0.0292
fl(fl(b^2) - fl(4ac))     = 0.1
absolute error            = 0.0708
relative error            = 2.424657534  (242.47%)

Interpretation:
  b^2  = 11.1556  -> 11.2  (3 significant digits)
  4ac  = 11.1264  -> 11.1  (3 significant digits)
  11.2 - 11.1 = 0.1, while the exact value is 0.0292
  This is a classic cancellation error.

[float]
```

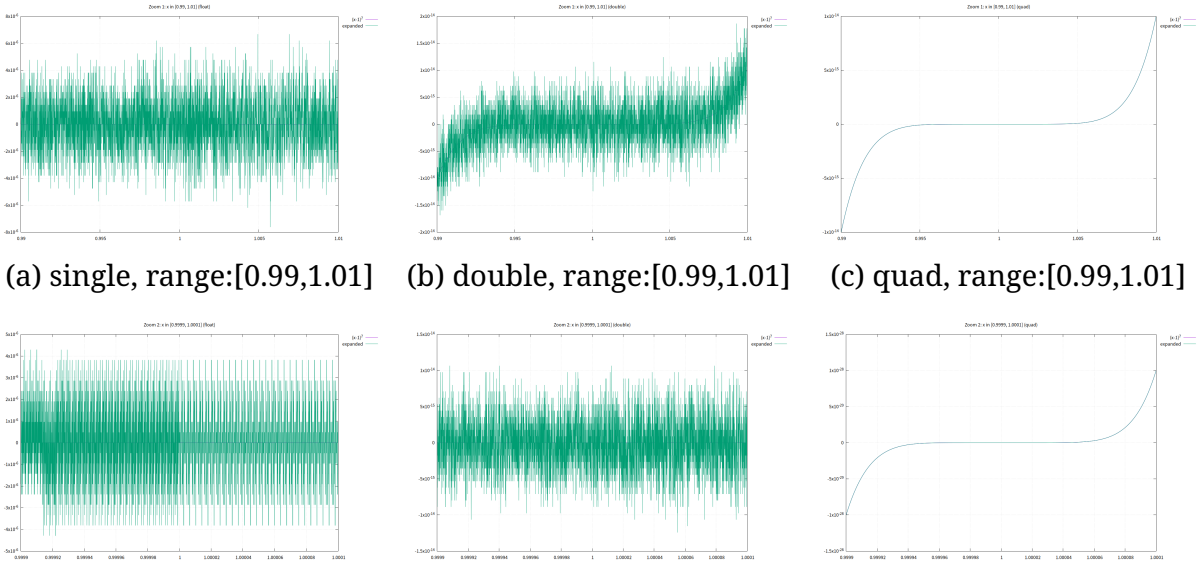
```
discriminant              = 0.0291996002
standard root 1           = -1.43888474
standard root 2           = -1.29882002
stable   root 1           = -1.43888474
stable   root 2           = -1.29882014

[double]
discriminant              = 0.029200000000001225
standard root 1           = -1.438885276600965
standard root 2           = -1.298819641431822
stable   root 1           = -1.438885276600965
stable   root 2           = -1.298819641431822

[quad: __float128]
discriminant             = 0.029199999999999999999999999999994601
standard root 1          = -1.4388852766009633702284561338216362 5
standard root 2          = -1.2988196414318235150174455055226259 9
stable   root 1          = -1.4388852766009633702284561338216362 5
stable   root 2          = -1.2988196414318235150174455055226259 9

Representative sample points near x = 1:
float  : x = 1.00100005, direct = 1.00032712e-21, expanded =
   4.76837158e-07
double : x = 1.0000000010000001, direct = 1.0000005791827407e-63,
   expanded = 3.5527136788005009e-15
quad   : x = 1.00000000000099999999999999999999, direct =
   9.99999999999999999999330873284919921e-85, expanded =
   -3.08148791101957736488956470813588371e-33

Generated files:
  poly_zoom1.csv   (x in [0.99,   1.01])
  poly_zoom2.csv   (x in [0.9999, 1.0001])
  plot.gp          (gnuplot script)
```



(a) single, range:[0.99,1.01] (b) double, range:[0.99,1.01] (c) quad, range:[0.99,1.01]

(d) single, range:[0.9999,1.0001] (e) double, range:[0.9999,1.0001] (f) quad, range:[0.9999,1.0001]

Figure 1: Comparison of the factored form $(x-1)^7$ and the expanded form under different precisions and zoom ranges.

4 Result Analysis

Comparison of the Quadratic Roots

From the output we can see that the two roots computed with high-precision `__float128` are approximately

$$x_1 \approx -1.4388852766009633702, \qquad x_2 \approx -1.2988196414318235150$$

Among them, the root with larger magnitude is almost identical under the standard formula and the stable formula, while the root with smaller magnitude is more sensitive to cancellation error. The stable formula is used here only as a reference for comparison. This is because, for the parameters in this problem, the evaluation of

$$\frac{-b+\sqrt{\Delta}}{2a}$$

contains the subtraction of two nearby numbers, which leads to a loss of significant digits.

The `float` results already show a visible difference in the second root between the standard formula and the stable formula. In `double` and `quad` precision, this difference becomes much smaller. This indicates that increasing

floating-point precision can alleviate the problem, but it does not fundamentally remove the instability of the algorithm itself.

Error Analysis of the Expanded Polynomial

Near $x = 1$, the true value of $(x-1)^7$ is extremely small. For example:

- for `float`, when $x \approx 1.00100005$, the factored form gives a value on the order of 10^{-21}, while the expanded form returns a value on the order of 10^{-7};
- for `double`, when $x \approx 1 + 10^{-9}$, the factored form is about 10^{-63}, while the expanded form is about 10^{-15};
- for `quad`, when $x \approx 1 + 10^{-12}$, the factored form is about 10^{-85}, while the expanded form is still only accurate to roughly the 10^{-33} scale.

This shows that although the expanded polynomial is analytically identical to $(x-1)^7$, it has already lost numerical stability in floating-point arithmetic. As x gets closer to 1, the positive and negative terms in the expansion cancel more severely, and the error becomes more pronounced.

The program also generates two zoomed data files:

- `poly_zoom1.csv`: corresponding to $x \in [0.99, 1.01]$;
- `poly_zoom2.csv`: corresponding to $x \in [0.9999, 1.0001]$.

After plotting these two intervals, we can observe that the factored form remains smooth and approaches 0 correctly, while the expanded form deviates significantly near $x = 1$, especially in lower precision.

Why the Stable Formula and the Factored Form Are Better

The two numerical phenomena in this problem both show that **mathematically equivalent expressions do not necessarily have the same numerical stability**.

1) for the quadratic equation, the stable formula reduces cancellation error by avoiding subtraction between nearby numbers;

2) for the polynomial, the factored form directly preserves the small quantity structure of the problem, whereas the expanded form writes a tiny result as the subtraction of several large terms, which greatly amplifies rounding error;

3) therefore, in numerical computation, we must care not only about whether a formula is mathematically correct, but also about whether the computational path is stable.

End of Report